

1. First Palindromic String in an Array

Aim: To find the first palindromic string in an array and analyze its time complexity.

Algorithm

1. Read an array of strings.
2. Check each string one by one.
3. Compare the string with its reverse.
4. If both are equal, it is a palindrome.
5. Print the first palindrome found.
6. If no palindrome exists, print an empty string.

Python Program

```
words = input("Enter strings separated by space: ").split()
```

```
found = False
```

```
for word in words:
```

```
    if word == word[::-1]:
```

```
        print("First Palindromic String:", word)
```

```
        found = True
```

```
        break
```

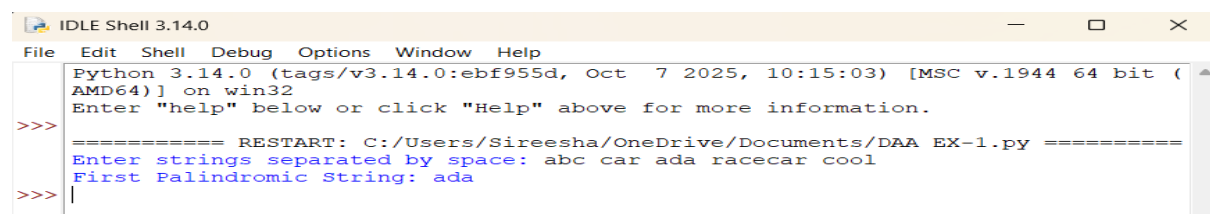
```
if not found:
```

```
    print("First Palindromic String: \"\")
```

Input

abc car ada racecar cool

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/DAA EX-1.py =====
Enter strings separated by space: abc car ada racecar cool
First Palindromic String: ada
>>>
```

Result

Thus, the Python program to find the first palindromic string in an array was successfully executed, and its time and space complexities were analyzed.

2. Common Elements Between Two Arrays

Aim: To find the common elements between two arrays and analyze its time complexity.

Algorithm

1. Read the first array.
2. Read the second array.
3. Compare every element of the first array with elements of the second array.
4. If an element is present in both arrays, add it to the result.
5. Print the common elements.

Python Program

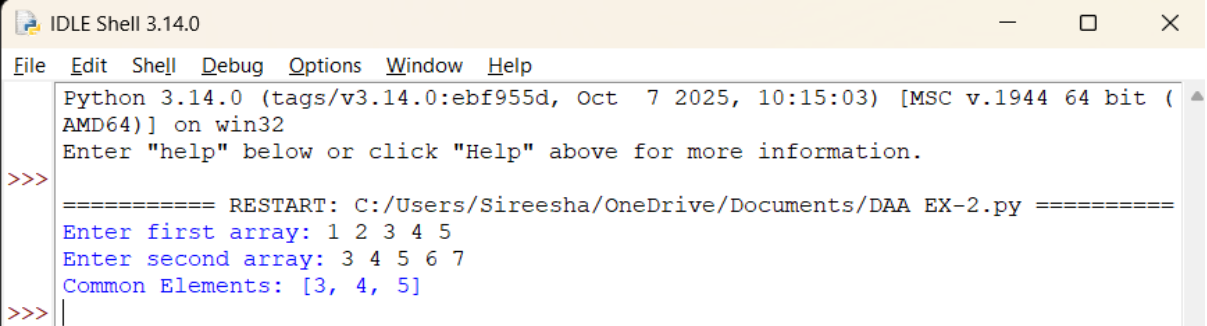
```
array1 = list(map(int, input("Enter first array: ").split()))
array2 = list(map(int, input("Enter second array: ").split()))
common = []
for x in array1:
    if x in array2 and x not in common:
        common.append(x)
print("Common Elements:", common)
```

Input

Enter first array: 1 2 3 4 5

Enter second array: 3 4 5 6 7

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/DAA EX-2.py =====
Enter first array: 1 2 3 4 5
Enter second array: 3 4 5 6 7
Common Elements: [3, 4, 5]
>>>
```

Result

Thus, the Python program to find the common elements between two arrays was successfully executed, and its time and space complexities were analyzed.

3. Bubble Sort with Big-O Analysis

Aim: To sort an array using Bubble Sort and analyze its time complexity using Big-O notation.

Algorithm

1. Read the array.
2. Compare adjacent elements.
3. If the first element is greater than the second, swap them.
4. Continue until the largest element reaches the end.
5. Repeat for all remaining elements.
6. Print the sorted array.

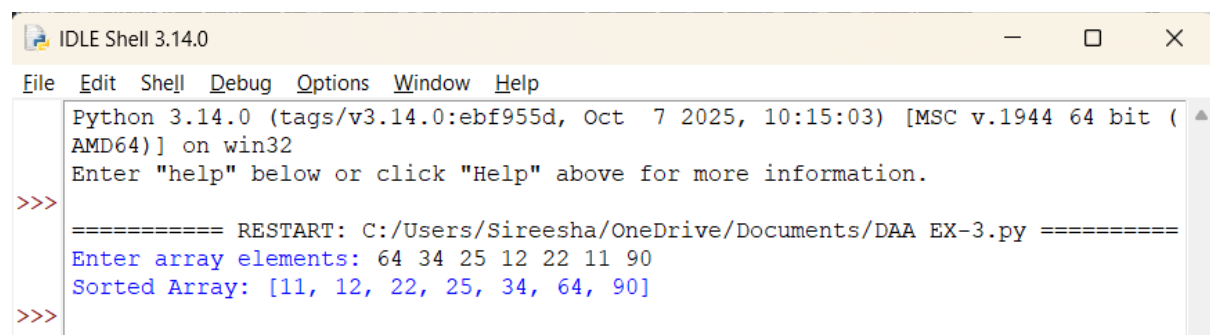
Python Program

```
arr = list(map(int, input("Enter array elements: ").split()))
n = len(arr)
for i in range(n):
    for j in range(0, n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
print("Sorted Array:", arr)
```

Input

Enter array elements: 64 34 25 12 22 11 90

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/DAA EX-3.py =====
Enter array elements: 64 34 25 12 22 11 90
Sorted Array: [11, 12, 22, 25, 34, 64, 90]
>>>
```

Result

Thus, the Python program to sort the array using Bubble Sort was successfully executed, and its Big-O time complexity was analyzed.

4. Binary Search with Big-O Analysis

Aim: To search for an element in a sorted array using Binary Search and analyze its time complexity.

Algorithm

1. Read a sorted array.
2. Set $low = 0$ and $high = n - 1$.
3. Find the middle element.
4. If the middle element equals the search element, return its position.
5. If the search element is smaller, search the left half.
6. Otherwise, search the right half.
7. Repeat until the element is found or the search range becomes empty.

Python Program

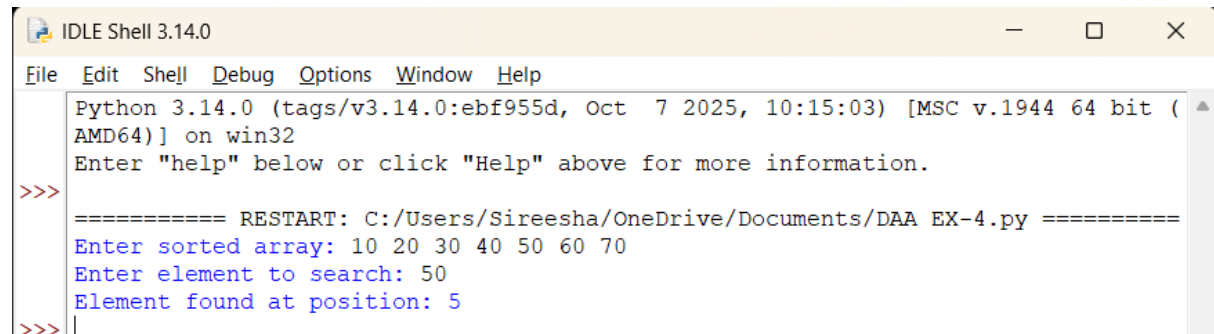
```
arr = list(map(int, input("Enter sorted array: ").split()))
key = int(input("Enter element to search: "))
low = 0
high = len(arr) - 1
found = False
while low <= high:
    mid = (low + high) // 2
    if arr[mid] == key:
        print("Element found at position:", mid + 1)
        found = True
        break
    elif key < arr[mid]:
        high = mid - 1
    else:
        low = mid + 1
if not found:
    print("Element not found")
```

Input

Enter sorted array: 10 20 30 40 50 60 70

Enter element to search: 50

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/Sireesha/OneDrive/Documents/DAA EX-4.py =====
Enter sorted array: 10 20 30 40 50 60 70
Enter element to search: 50
Element found at position: 5
>>>
```

Result

Thus, the Python program to search for an element using Binary Search was successfully executed, and its Big-O time complexity was analyzed.